

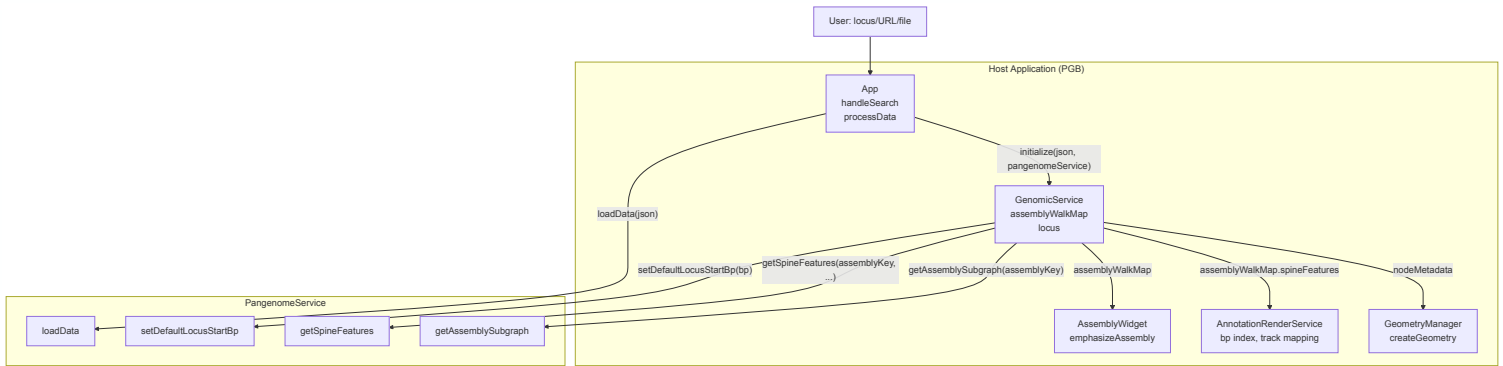
PGB ↔ PangenomeService Interaction Diagrams

Interaction flows for how **PangenomeService** is used in the PGB (Pan Genome Browser) application. PangenomeService loads pangenome graph data, computes assembly walks, spine features, and subgraphs — all consumed by GenomicService and downstream components for visualization and annotation.

1. Architecture Overview

PangenomeService is the **graph computation engine**. The host application loads JSON, passes it to PangenomeService, and uses the resulting spine features and subgraphs for 3D rendering, assembly emphasis, and annotation tracks.

Component	Role	Owns
App	Orchestrator	Loads JSON, calls <code>loadData</code> , triggers <code>genomicService.initialize</code>
PangenomeService	Graph engine	Nodes, edges, walks, spine features, assembly subgraphs
GenomicService	Consumer	<code>assemblyWalkMap</code> (spine + subgraph per assembly), locus, node metadata
AssemblyWidget	UI	Assembly selector; uses <code>assemblyWalkMap</code> for emphasis
AnnotationRenderService	Rendering	Uses <code>assemblyWalkMap.spineFeatures</code> for bp index and track mapping
GeometryManager	3D geometry	Uses <code>genomicService.nodeMetadata</code> (derived from JSON, not PangenomeService)

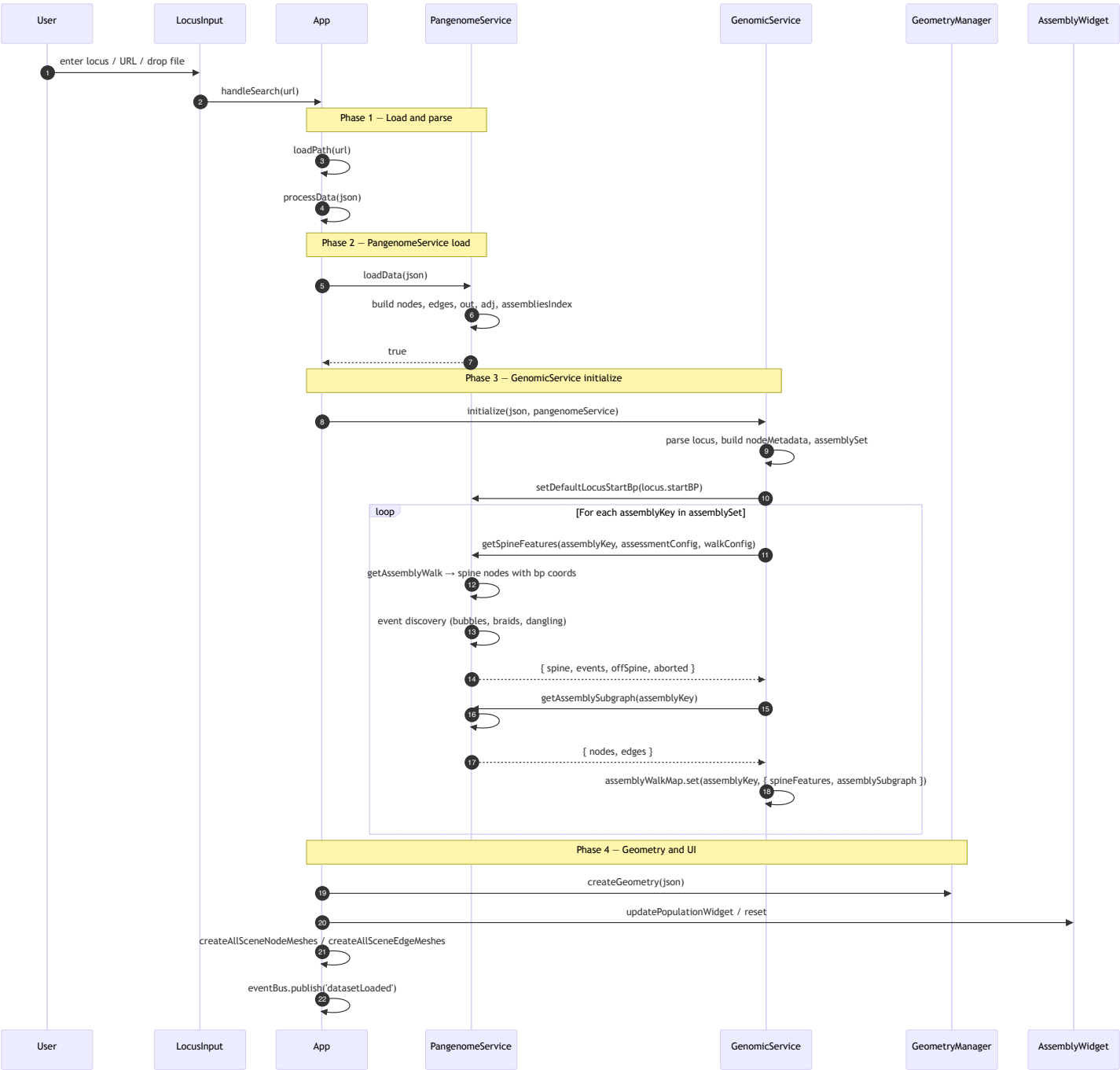


2. Data Load — Full Sequence

When the user enters a locus, URL, or drops a JSON file, the full pipeline runs. PangenomeService is invoked during `processData`.

Entry points: (1) LocusInput (locus string, URL, or local file path) → `app.handleSearch(url)`; (2) Drag-and-drop of JSON file → container drop handler → `app.processData(json)` directly.

JSON file → container drop handler → `app.processData(json)` directly.



Key Points

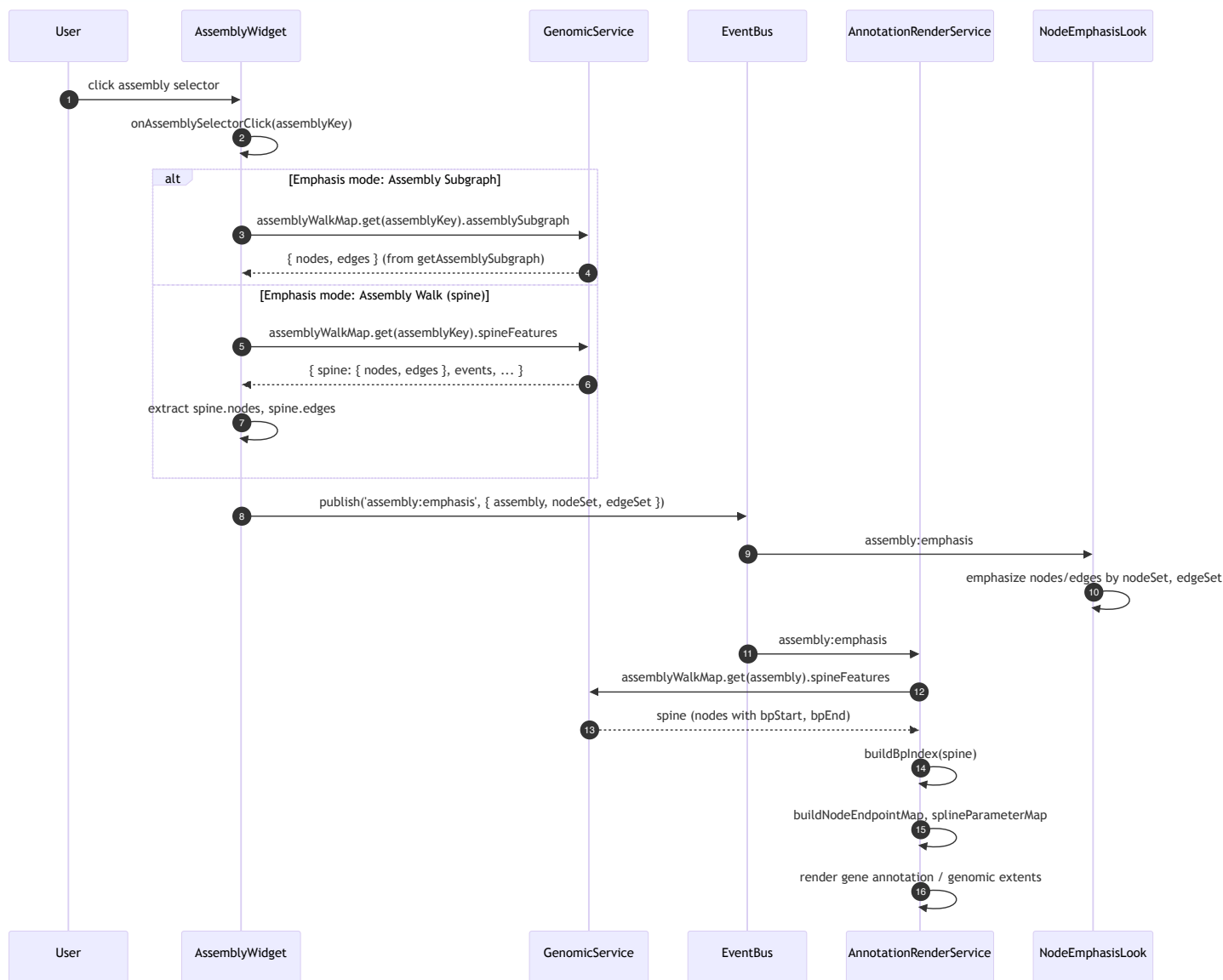
Aspect	Detail
Single loadData call	App calls <code>pangenomeService.loadData(json)</code> once per dataset
Per-assembly iteration	GenomicService loops over <code>assemblySet</code> and calls <code>getSpineFeatures</code> + <code>getAssemblySubgraph</code> for each
Cached in assemblyWalkMap	Results are stored in <code>genomicService.assemblyWalkMap</code> ; downstream components read from there

No direct
PangenomeService after
init

AssemblyWidget and AnnotationRenderService use
`genomicService.assemblyWalkMap` , not PangenomeService directly

3. Assembly Emphasis — User Clicks Assembly Selector

When the user clicks an assembly in the Assembly Widget, the app emphasizes nodes/edges. Data comes from `assemblyWalkMap` , which was populated by PangenomeService during init.

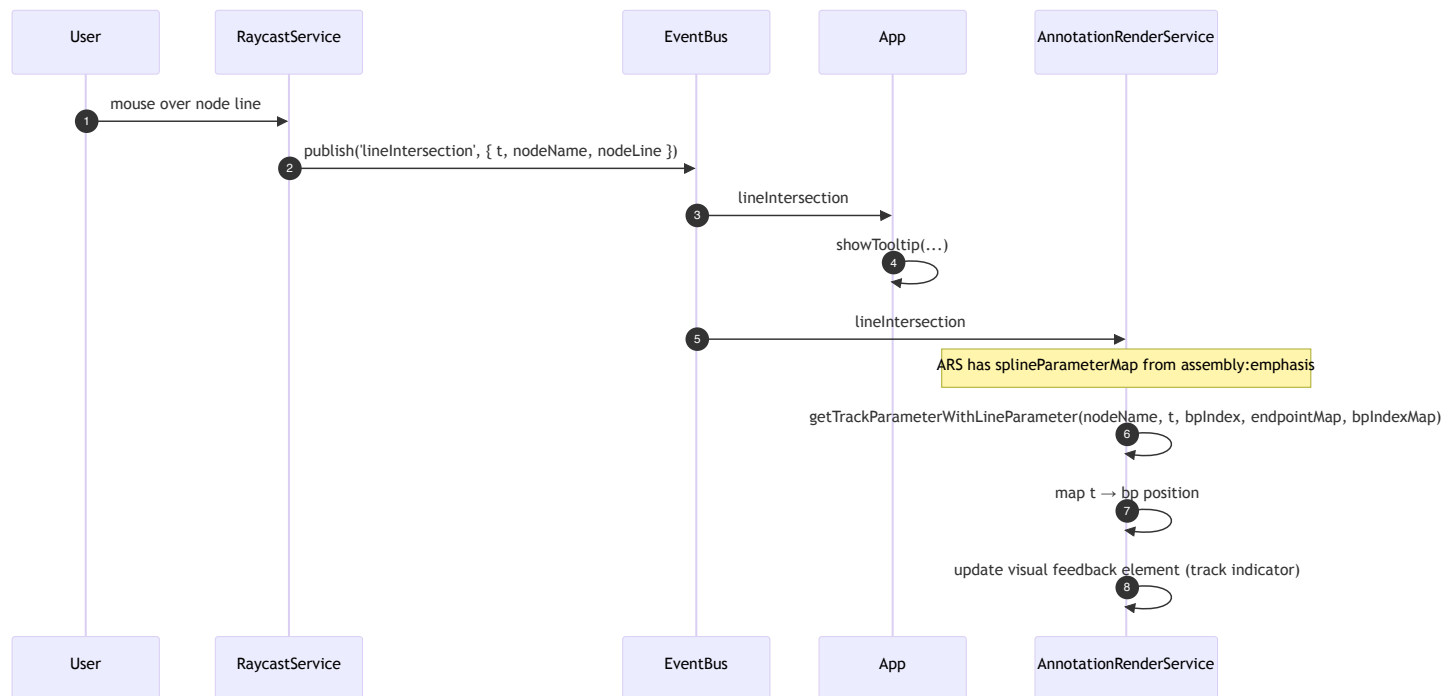


Data Source by Mode

Mode	Source	PangenomeService method
Assembly Subgraph	<code>assemblyWalkMap.get(key).assemblySubgraph</code>	<code>getAssemblySubgraph(assemblyKey)</code>
Assembly Walk	<code>assemblyWalkMap.get(key).spineFeatures.spine</code>	<code>getSpineFeatures(...).spine</code>

4. Annotation Track — Line Intersection (Hover)

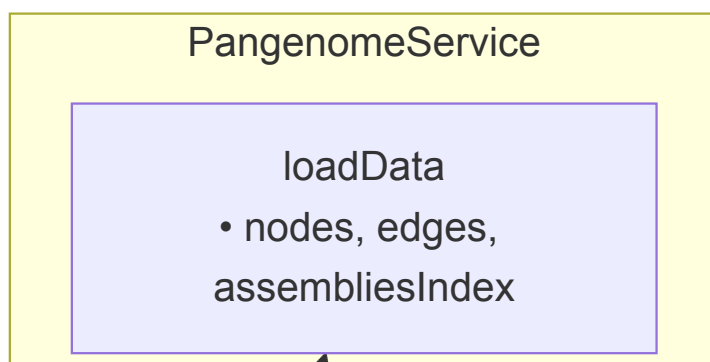
When the user hovers over a node line, the annotation track shows a vertical indicator at the corresponding base-pair position. The bp mapping relies on spine data from PangenomeService (via GenomicService).

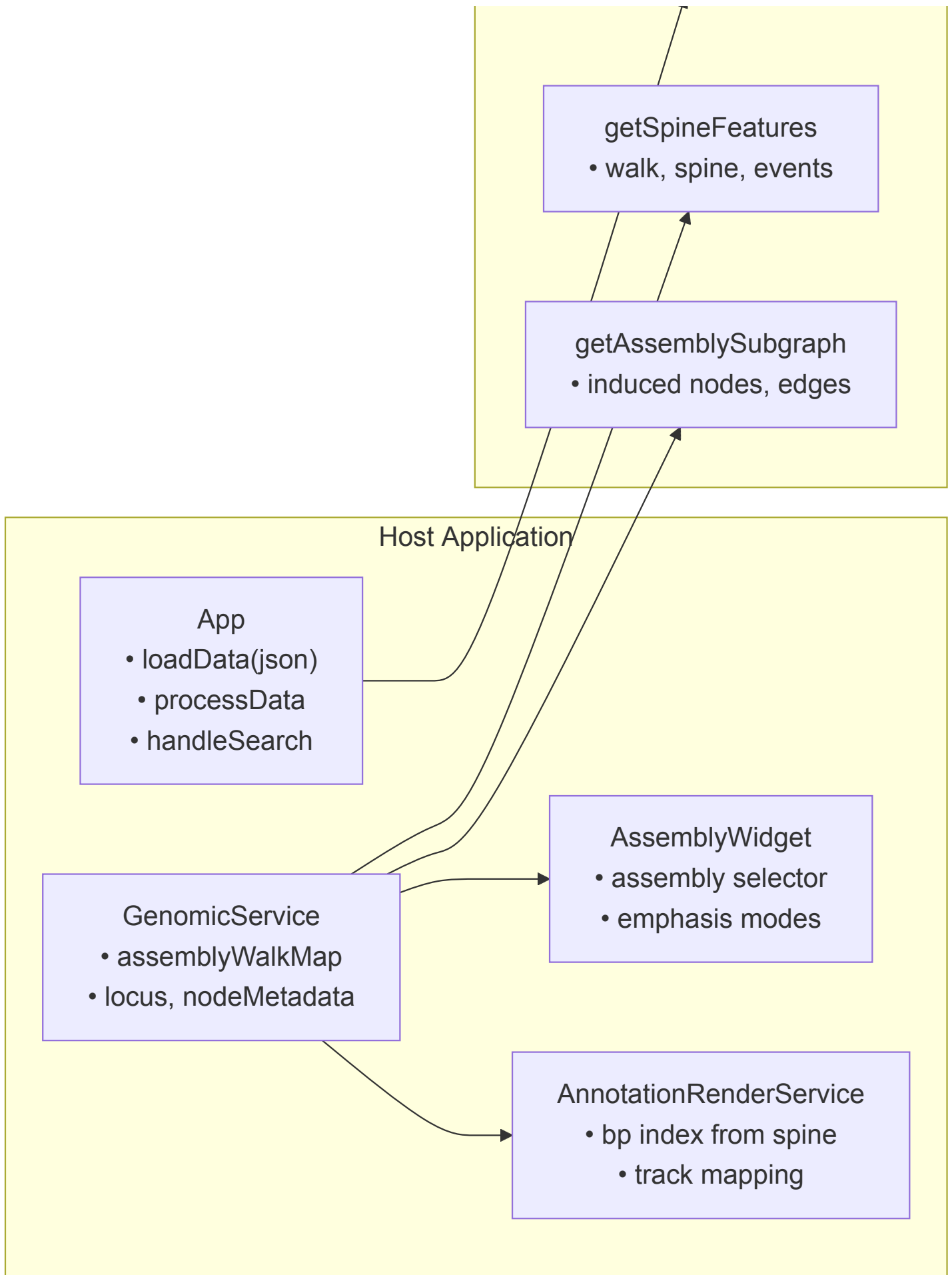


Spine Data Flow for Annotations

```
PangenomeService.getSpineFeatures()
  → spine.nodes: [{ id, bpStart, bpEnd, lengthBp }, ...]
  → GenomicService.assemblyWalkMap
  → AnnotationRenderService.buildBpIndex(spine)
  → bpIndex, bpIndexMap, endpointMap
  → getTrackParameterWithLineParameter(t, ...) → bp
```

5. Component Responsibilities





Component	Lives in	PangenomeService usage
App	app.js	<code>loadData(json)</code> ; passes pangenomeService to genomicService.initialize
GenomicService	genomicService.js	<code>setDefaultLocusStartBp</code> , <code>getSpineFeatures</code> , <code>getAssemblySubgraph</code> ; caches in assemblyWalkMap
AssemblyWidget	widgets/assemblyWidget.js	Reads <code>assemblyWalkMap</code> (no direct PangenomeService calls)
AnnotationRenderService	annotationRenderService.js	Reads <code>assemblyWalkMap.spineFeatures</code> for spine (no direct PangenomeService calls)
GeometryManager	geometryManager.js	Uses genomicService.nodeMetadata (from JSON); no PangenomeService

6. PangenomeService API Used by Host

Method	Caller	When
<code>loadData(json)</code>	App	Once per dataset load (processData)
<code>setDefaultLocusStartBp(bp)</code>	GenomicService	During initialize, after parsing locus
<code>getSpineFeatures(assemblyKey, assessOpts, walkOpts)</code>	GenomicService	Once per assembly during initialize
<code>getAssemblySubgraph(assemblyKey)</code>	GenomicService	Once per assembly during initialize

Methods Not Used by Host

Method	Purpose
<code>listAssemblyKeys()</code>	Returns assembly keys; host derives assemblySet from JSON node metadata instead
<code>getAssemblyWalk()</code>	Used internally by <code>getSpineFeatures</code> ; host does not call directly
<code>getDefaultLocusStartBp()</code>	Getter; host does not read back
<code>getAssemblySubgraph(..., { includeAdj, includeDirectedAdj })</code>	Host uses default options only

7. Data Flow Summary

JSON (node, edge, sequence, locus)

```

graph TD
    A[PangenomeService.loadData(json)] --> B[nodes Map, edges Map, out, adj, assembliesIndex]
    B --> C[GenomicService.initialize(json, pangenomeService)]
    C --> D[setDefaultLocusStartBp(locus.startBP)]
    C --> E[for each assemblyKey:
    |   getSpineFeatures(assemblyKey, ...)
    |       |   getAssemblyWalk() → path
    |       |   spine nodes with bpStart, bpEnd
    |       |   events (bubbles, braids, dangling)
    |   getAssemblySubgraph(assemblyKey)
    |       |   induced { nodes, edges }
    |   assemblyWalkMap.set(assemblyKey, { spineFeatures, assemblySubgraph })]
    E --> F[assemblyWalkMap]
    F --> G[AssemblyWidget.emphasizeAssembly()
    |   |   spineFeatures.spine (Assembly Walk mode)
    |   |   assemblySubgraph (Assembly Subgraph mode)]
    G --> H[AnnotationRenderService.handleAssemblyEmphasis()
    |   |   spineFeatures.spine → buildBpIndex, splineParameterMap]

```

8. Lifecycle — Clear on New Load

When a new dataset is loaded, `clearCurrentData` runs before `processData`. GenomicService clears its maps (including assemblyWalkMap). PangenomeService is not explicitly cleared; `loadData` overwrites its internal graph.



Syntax error in text
mermaid version 11.10.1